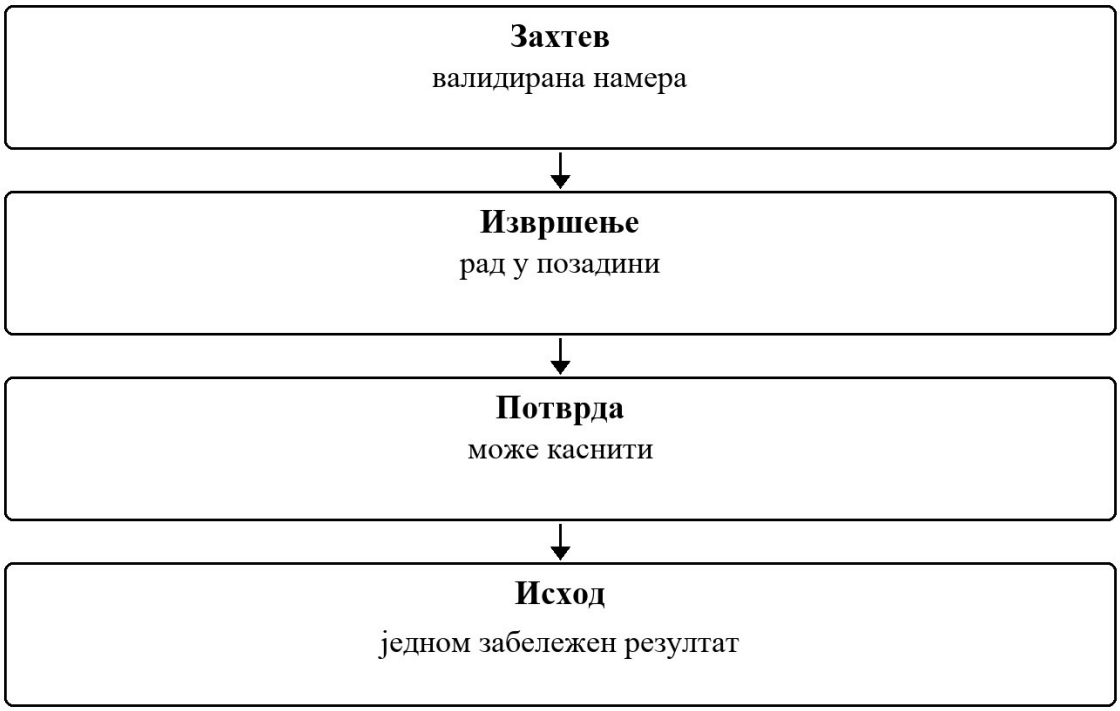
Команде, послови, поновни покушај и идемпотентност

Основе дистрибуираног програмирања · 06

Команда је захтев за промену света, а не обичан позив методе. Након слања, систем може чекати, понављати покушај или добити касну потврду. Зато се животни циклус команде моделује експлицитно и разликује од локалне намере корисника.

Поновни покушај повећава вероватноћу успеха, али без идемпотентности може поновити пословни ефекат. Леасе и ограничено власништво над послом спречавају да два воркера истовремено доврше исту радњу након неусаглашеног погледа на стање.



Слика 1. Како систем напредује када потврда није тренутна ни сигурна

Дистрибуирана својства настају у односу између компоненти, времена и граница одговорности.

Фокус вежбе

Материјал објашњава дистрибуирани проблем, последице избора и границе гаранције коју систем може да пружи.

6.1 Поновни покушај не сме мењати значење радње

- Статус команде описује знање система, не нагађање о удаљеном свету.
- Идемпотенци кључ везује поновљене покушаје за исти пословни резултат.
- Леасе има рок и штити систем од закашњелог рада претходног власника.

Питање за дискусију

Која би последица дуплог извршења била неприхватљива и који идентитет операције спречава такав исход?

6.2 Како се ова тема види у развоју

Тим раздваја пријем захтева, обраду у позадини и приказ исхода. Затим тестира прекид између њих: поновљени захтев, пад воркера и касна потврда не смеју произвести нову пословну радњу нити поништити новије стање.

Честе грешке које треба препознати

- Поновни покушај без границе и без разлога за престанак.
- Две обраде које обе верују да су власници посла.
- Касна потврда која преписује завршено стање.

Веза са остатком градива

Овај принцип се надовезује на уговоре, отказ сценарије, могућност посматрања система и тестирање. Иста логика важи без обзира на конкретну технологију или домен система.

6.3 Пример: поновљен захтев за исту радњу

Клијент не добија одговор на време и шаље исти захтев поново. Систем жели да буде отпоран на прекид, па други захтев не сме аутоматски сматрати новом намером. Идемпотенци кључ повезује поновљене покушаје са истим пословним резултатом, док статус животног циклуса јасно одваја прихватање, обраду, чекање и завршетак.

- Поновни покушај користи исти идентитет операције када намера корисника није промењена.
- Воркер који преузима посао има ограничено време власништва и не може завршити рад након истека.
- Касна потврда не сме преписати новије, завршено стање.

Како се сценарио анализира

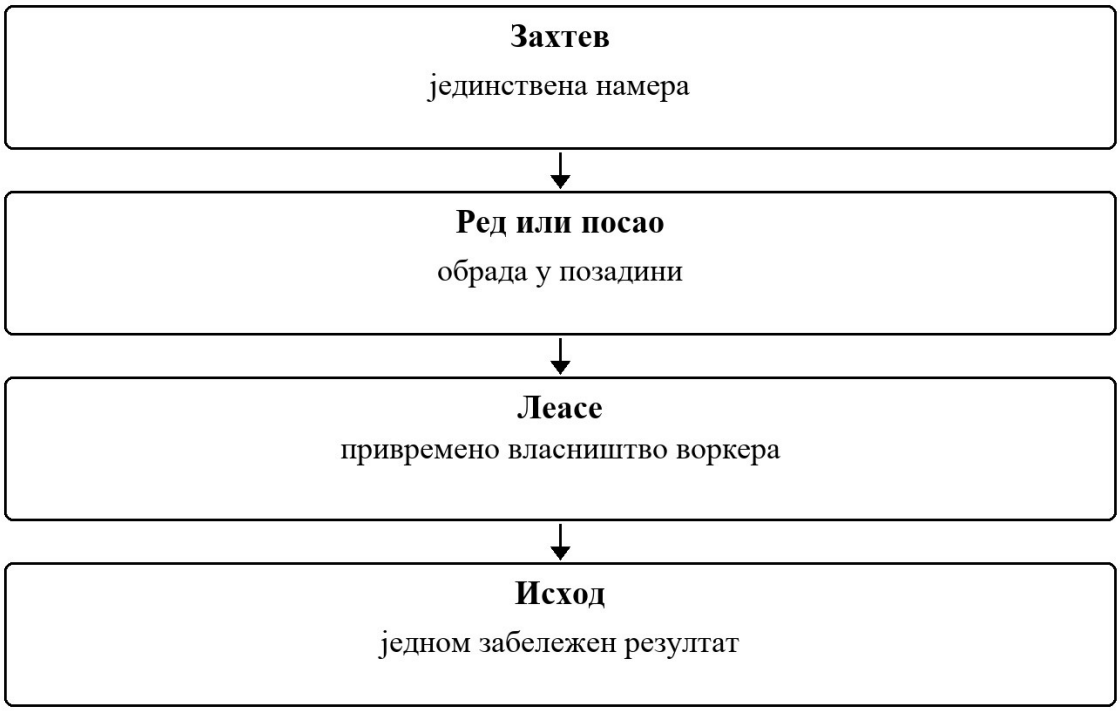
Најпре раздвојите оно што систем зна од онога што претпоставља. Затим одредите стабилан идентитет операције и видљив исход који се може проверити.

6.4 Од принципа до доказа

Дистрибуирани принцип је користан тек када систем има јасан, проверљив одговор на његов сценарио отказа. Добар доказ не показује само да је нормалан ток успео, већ и да кашњење, дупликат, прекид или касна порука не мењају пословно значење операције.

- Именовати стање које је ауторитативно и компоненту која има право да га мења.
- Навести шта се дешава када одговор изостане или стигне након истека рока.
- Понављати контролисани сценарио док се исход може објаснити и независно проверити.

6.5 Шема поуздане команде



Слика 2. Шема поуздане команде

Поновни покушај повећава шансу да се радња заврши, док идемпотентност чува њено значење.

Шему користите као кратку проверу: за сваки корак треба умети навести власника одлуке, поруку или стање које се преносе и сценарио отказа који може променити ток.

6.6 Упоредни преглед елемената тока

Табела 1. Елемент и Улога у току

Елемент	Улога у току	Питање које треба поставити
Захтев	валидирана намера	Шта се дешава када захтев није доступан или даје застарелу информацију?
Извршење	рад у позадини	Шта се дешава када извршење није доступан или даје застарелу информацију?
Потврда	може каснити	Шта се дешава када потврда није доступан или даје застарелу информацију?
Исход	једном забележен резултат	Шта се дешава када исход није доступан или даје застарелу информацију?

Поређење појмова

Сваки елемент тока решава други део проблема. Стабилан дизајн не пребацује одговорност једног елемента на други само зато што је то краткорочно једноставније.

6.7 Питања за анализу дистрибуираног понашања

Када се анализира дистрибуирани ток, није довољно описати само срећан пут. Потребно је именовати стање које се мења, границу одговорности, поруку или време које може закаснити и исход који систем мора сачувати упркос неизвесности.

1. Статус команде описује знање система, не нагађање о удаљеном свету.

2. Идемпотенци кључ везује поновљене покушаје за исти пословни резултат.

3. Леасе има рок и штити систем од закашњелог рада претходног власника.

4. Који контролисани сценарио би показао да се пословно значење операције не мења при кашњењу, дупликату или прекиду?

Питање за вежбу

Која би последица дуплог извршења била неприхватљива и који идентитет операције спречава такав исход?

6.8 Извршиви пример

Пример води команду ка удаљеном уређају кроз неизвесност. Слање нема повратну вредност: потврда стиже одвојено, касни или не стигне. Зато команда има стабилан идентификатор, поновно слање носи исти идентификатор, уређај га извршава једном, а потврда која стигне после истека и даље исправља стање.

Пример 1. Покретање демонстрације и тестова

```
cd examples/odp-dotnet/vezba-06-komande-idempotentnost
dotnet run --project src/Odp.Vezba06.ConsoleUi
dotnet test Odp.Vezba06.sln
```

Правило у коду

Пример 2. examples/odp-dotnet/vezba-06-komande-idempotentnost/src/Odp.Vezba06.Domain/Commands/CommandLifecycle.cs

```
public (DeviceCommand Command, string Code) Acknowledge(DeviceCommand command) =>
    command.State switch
    {
        CommandState.Sent =>
            (command with { State = CommandState.Acknowledged },
CommandCodes.Acknowledged),
        CommandState.TimedOut =>
            (command with { State = CommandState.Acknowledged },
CommandCodes.LateAckReconciled),
        _ => (command, CommandCodes.AckDuplicate)
    };
```

Тест који задаје квар

Пример 3. examples/odp-dotnet/vezba-06-komande-idempotentnost/tests/Odp.Vezba06.Tests/Application/CommandFlowTests.cs

```
[Test]
public void Run_WhenAckIsMissing_ResendsTheSameCommandAndTheDeviceExecutesItOnce()
{
    _dispatch.Dispatch("cmd-1", "antena-1", "rotate:120");
    _clock.Advance(TimeSpan.FromSeconds(12));

    var outcomes = _retry.Run();

    using (Assert.EnterMultipleScope())
    {
        Assert.That(outcomes, Is.EqualTo(new[] { new RetryOutcome("cmd-1",
CommandCodes.Retried) }));
        Assert.That(_device.Deliveries, Is.EqualTo(2));
        Assert.That(_device.ExecutedCommandIds, Is.EqualTo(new[] { "cmd-1" }));
    }
}
```

Испис демонстрације

Пример 4. дотнет рун --пројецт src/Odp.Vezba06.ConsoleUi

```
operator šalje komandu -> command_sent (pokušaj 1)
operator ponavlja isti zahtev -> command_already_dispatched (pokušaj 1)
potvrda nije stigla 12 s -> cmd-7: command_retried
potvrda nije stigla još 12 s -> cmd-7: attempts_exhausted
zakasnela potvrda -> late_ack_reconciled
ista potvrda ponovo -> ack_duplicate
```

```
Uređaj je primio 2 isporuke, a izvršio 1 komandu.
```

Шта треба уочити

- Поновљени захтев са истим `CommandId` не шаље команду други пут.
- После истека се шаље иста команда; уређај је прима два пута, а извршава једном.
- Потврда која стигне после одустајања исправља стање (`late_ack_reconciled`).

Задатак уз пример

Додати одлагање између покушаја (`RetryDelay`) које расте са бројем покушаја. Правило додати у `CommandLifecycle` и тестом показати да се поновно слање не дешава пре истека одлагања.

Рад на вежби — примена на пројекат

На додељеној пројектној целини применити принцип из теме „Команде, послови, поновни покушај и идемпотентност". Идентификовати конкретну дистрибуирану одлуку, имплементирати је и припремити поновљив сценарио отказа и тест који тим може да покаже на пројектној контролној тачки.

ПЗ Тестирање и мануал-цоре-баселине

Контролна тачка

Ова тачка заокружује претходне теме. Организациони детаљи и захтеви налазе се у посебном одељку контролних тачака.